Question 1

1. In our implementation, Alice and Bob increment their Diffie-Hellman ratchets every time they exchange messages. Could the protocol be modified to have them increment the DH ratchets once every ten messages without compromising confidentiality against an eavesdropper (i.e., semantic security)? Why or why not?

Yes, updating the DH ratchet once every ten messages would still preserve semantic security against a passive eavesdropper. The symmetric-key ratchet generates a fresh, pseudorandom message key using HMAC-SHA256 for every message, so messages remain encrypted under distinct keys even without a new DH exchange. However, less frequent DH ratcheting would delay break-in recovery and increase the number of future messages potentially exposed after a key compromise.

Question 2

2. What if they never update their DH keys at all? Please explain the security consequences of this change with regard to forward secrecy and break-in recovery.

Without updating DH keys, the symmetric-key ratchet would still provide some forward secrecy because previously deleted chain keys cannot feasibly be derived from current chain keys. However, if an attacker compromises the retained initial DH private key, they could reconstruct the initial shared secret and derive past message keys, defeating forward secrecy for those messages. Furthermore, break-in recovery would be lost because an attacker who compromises the current chain keys could derive all subsequent message keys without any fresh DH secret being introduced.

Question 3

3. The message reporting feature included in our messaging scheme is not adequate for use on an actual messaging platform. What is one shortcoming of our approach?

One shortcoming is that the reporting system cannot verify whether a reported message was actually sent by the alleged sender. Since the reporting client supplies both the sender's name and message contents, a malicious user could fabricate a message, attribute it to another user, and submit it as a legitimate report. The encryption protects the report's confidentiality but does not establish its authenticity.

Question 4

4. Our messaging system relies on the platform to verify the authenticity of users' public keys, but we are also trying to give users confidentiality from the platform.

Using this partial trust we place in it, how could a malicious platform learn the contents of a message one user is sending to another?

A malicious platform could perform a man-in-the-middle attack by replacing Bob's public key in his certificate with a public key controlled by the platform and signing the fraudulent certificate using its legitimate signing key. Alice would accept this certificate as authentic and establish an encrypted session with the platform rather than Bob. The platform could then decrypt Alice's messages, read their contents, and re-encrypt them using Bob's actual public key before forwarding them.

Question 5
5. How do end-to-end encrypted messaging apps help users avoid this problem? Feel free to look this up or explore one of these messaging apps to answer this question

End-to-end encrypted messaging applications such as Signal allow users to independently verify their contacts' encryption keys using safety numbers or QR codes, which can be compared through a trusted channel outside the messaging platform. Signal also alerts users when a contact's safety number changes, helping detect potentially malicious key substitutions. WhatsApp additionally uses key transparency, maintaining an auditable directory of public keys to help detect unauthorized changes. These mechanisms reduce reliance on the platform as the sole authority for public-key authenticity.